# 188--Lab - Administración de recursos mediante el etiquetado

En esta práctica se muestra como inspeccionar las etiquetas asignadas a instancias EC2 y usar scripts para iniciar y terminar instancias con base en las etiquetas.

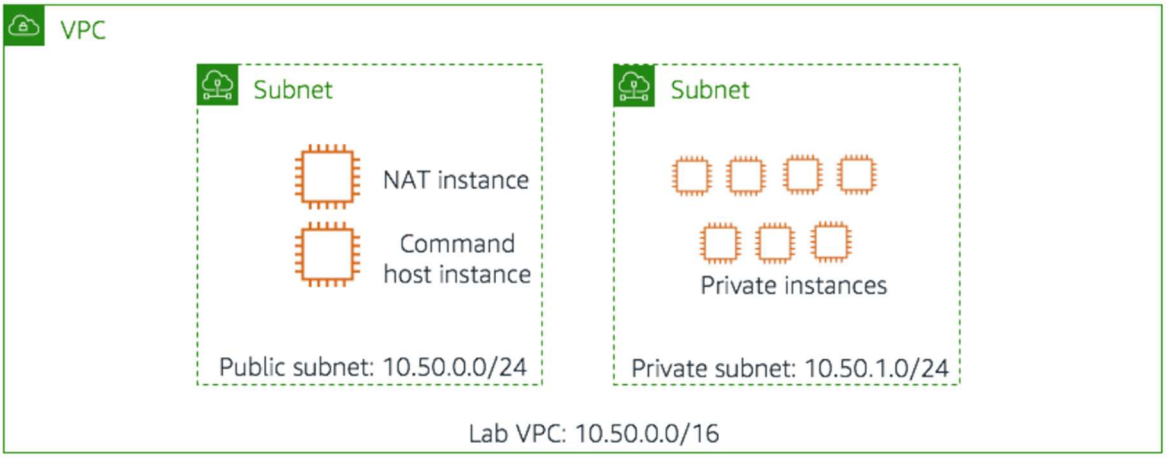
De igual forma se muestra como terminar aquellas instancias que no cumplan con etiquetas específicas.

## Escenario

Las instancias tienen asignadas las siguientes etiquetas:

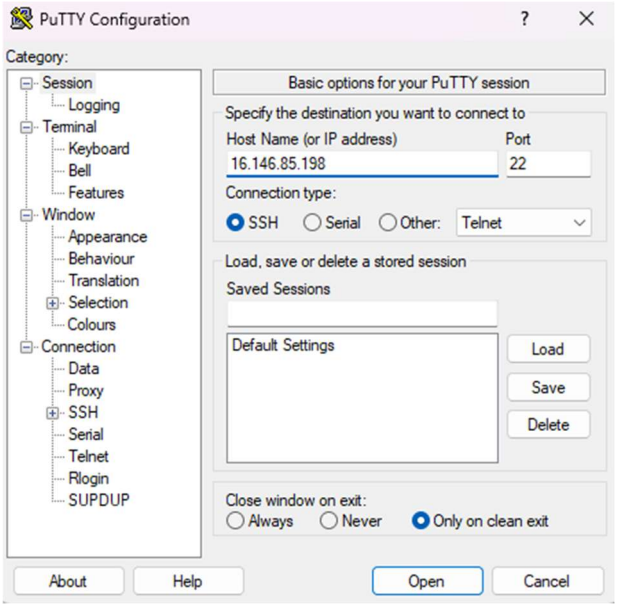
| Tag Name | content |
|---|---|
| Project | The project that the instance belongs to. The instances in this lab belong to one of two projects: **ERPSystem** and **Experiment1**. |
| Version | The version of the project that this instance belongs to. All Version tags are currently set to 1.0. |
| Environment | One of three values: **development**, **staging**, or **production**. |

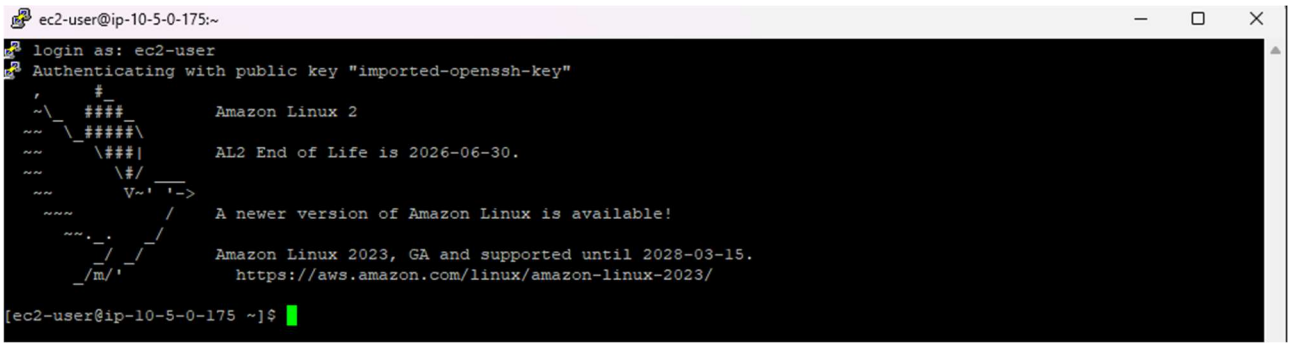
La arquitectura actual es la siguiente:




## Usar etiquetas para administrar recursos

Nos conectamos a la instancia Command Host mediante SSH utilizando Putty




Buscamos la instancias que tengan la etiqueta Project=ERPSystem

```
aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem"
```

```
[ec2-user@ip-10-5-0-175 ~]$ aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem"
{
    "Reservations": [
        {
            "Instances": [
                {
                    "Monitoring": {
                        "State": "disabled"
                    },
                    "PublicDnsName": "",
                    "State": {
                        "Code": 16,
                        "Name": "running"
                    },
                    "EbsOptimized": false,
                    "LaunchTime": "2026-02-16T21:11:35.000Z",
                    "PrivateIpAddress": "10.5.1.242",
                    "ProductCodes": [],
                    "VpcId": "vpc-0170241af1d7de2f9",
                    "CpuOptions": {
                        "CoreCount": 1,
                        "ThreadsPerCore": 2
                    },
                    "StateTransitionReason": "",
                    "InstanceId": "i-0eb8ec00fc8aa4df7",
                    "EnaSupport": true,
                    "ImageId": "ami-015c7ae0e2a6d8153",
                    "PrivateDnsName": "ip-10-5-1-242.us-west-2.compute.internal",
```

```
            "StateTransitionReason": "",
            "InstanceId": "i-0eb8ec00fc8aa4df7",
            "EnaSupport": true,
            "ImageId": "ami-015c7ae0e2a6d8153",
            "PrivateDnsName": "ip-10-5-1-242.us-west-2.compute.internal",
            "KeyName": "vockey",
            "SecurityGroups": [
                {
                    "GroupName": "default",
                    "GroupId": "sg-0a1054516ba7855aa"
                }
            ],
            "ClientToken": "f0569070-6c04-f31c-d1e6-5b9a02e26d9c",
            "SubnetId": "subnet-0f66e9e204fff4b34",
            "InstanceType": "t3.micro",
            "CapacityReservationSpecification": {
                "CapacityReservationPreference": "open"
            },
```

```
            "Tags": [
                {
                    "Value": "portal",
                    "Key": "Application"
                },
                {
                    "Value": "app server",
                    "Key": "Name"
                },
                {
                    "Value": "staging",
                    "Key": "Environment"
                },
                {
                    "Value": "ERPSystem",
                    "Key": "Project"
                },
                {
                    "Value": "HR",
                    "Key": "Department"
                },
                {
                    "Value": "1.0",
                    "Key": "Version"
                },
```


Hay 7 instancias con la etiqueta especificada

Ahora obtenemos solo los IDs de estas instancias

```
aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem" --query 'Reservations[*].Instances[*].InstanceId'
```

```
[ec2-user@ip-10-5-0-175 ~]$ aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem"
--query 'Reservations[*].Instances[*].InstanceId'
[
    [
        "i-0eb8ec00fc8aa4df7"
    ],
    [
        "i-0e9426023cbc3d32c"
    ],
    [
        "i-0e1c528b2a18d9bc1"
    ],
    [
        "i-0f87dfd3ae1aedbb2"
    ],
    [
        "i-0f328cea8169f1a11"
    ],
    [
        "i-0467ec8d7b64aa588"
    ],
    [
        "i-04732903097bfe237"
    ]
]
```

El asterisco significa que el comando itera sobre todas las instancias

Ahora obtenemos los IDs y zonas de disponibilidad de cada instancia

```
aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem" --query
'Reservations[*].Instances[*].{ID:InstanceId,AZ:Placement.AvailabilityZone}'
```

```
[ec2-user@ip-10-5-0-175 ~]$ aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem"
--query 'Reservations[*].Instances[*].{ID:InstanceId,AZ:Placement.AvailabilityZone}'
[
    [
        {
            "AZ": "us-west-2a",
            "ID": "i-0eb8ec00fc8aa4df7"
        }
    ],
    [
        {
            "AZ": "us-west-2a",
            "ID": "i-0e9426023cbc3d32c"
        }
    ],
    [
        {
            "AZ": "us-west-2a",
            "ID": "i-0e1c528b2a18d9bc1"
        }
    ],
    [
        {
            "AZ": "us-west-2a",
            "ID": "i-0f87dfd3ae1aedbb2"
        }
    ],
    [
        {
            "AZ": "us-west-2a",
            "ID": "i-0f328cea8169f1a11"
        }
    ],
    [
        {
            "AZ": "us-west-2a",
            "ID": "i-0467ec8d7b64aa588"
        }
    ],
    [
        {
            "AZ": "us-west-2a",
            "ID": "i-04732903097bfe237"
        }
    ]
]
```

En este comando ejecutamos un alias para los resultados (AZ y ID).

Obtenemos los IDs, zonas de disponibilidad y proyecto de cada instancia

```
aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem" --query
'Reservations[*].Instances[*].{ID:InstanceId,AZ:Placement.AvailabilityZone,Project:T
ags[?Key==`Project`] | [0].Value}'
```

```
ec2-user@ip-10-5-0-175 ~]$ aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem"
-query 'Reservations[*].Instances[*].{ID:InstanceId,AZ:Placement.AvailabilityZone,Project:Tags[?Key
==`Project`] | [0].Value}'

    [
        {
            "Project": "ERPSystem",
            "AZ": "us-west-2a",
            "ID": "i-0eb8ec00fc8aa4df7"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "AZ": "us-west-2a",
            "ID": "i-0e9426023cbc3d32c"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "AZ": "us-west-2a",
            "ID": "i-0e1c528b2a18d9bc1"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "AZ": "us-west-2a",
            "ID": "i-0f87dfd3ae1aedbb2"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "AZ": "us-west-2a",
            "ID": "i-0f328cea8169f1a11"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "AZ": "us-west-2a",
            "ID": "i-0467ec8d7b64aa588"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "AZ": "us-west-2a",
            "ID": "i-04732903097bfe237"
```

Obtenemos también el valor de las etiquetas Environment y Version

```
aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem" --query
'Reservations[*].Instances[*].{ID:InstanceId,AZ:Placement.AvailabilityZone,Project:T
ags[?Key==`Project`] | [0].Value,Environment:Tags[?Key==`Environment`] |
[0].Value,Version:Tags[?Key==`Version`] | [0].Value}'
```

```
[ec2-user@ip-10-5-0-175 ~]$ aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem"
--query 'Reservations[*].Instances[*].{ID:InstanceId,AZ:Placement.AvailabilityZone,Project:Tags[?Key
==`Project`] | [0].Value,Environment:Tags[?Key==`Environment`] | [0].Value,Version:Tags[?Key==`Versi
on`] | [0].Value}'
[
    [
        {
            "Project": "ERPSystem",
            "Environment": "staging",
            "AZ": "us-west-2a",
            "Version": "1.0",
            "ID": "i-0eb8ec00fc8aa4df7"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "Environment": "development",
            "AZ": "us-west-2a",
            "Version": "1.0",
            "ID": "i-0e9426023cbc3d32c"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "Environment": "development",
            "AZ": "us-west-2a",
            "Version": "1.0",
            "ID": "i-0e1c528b2a18d9bc1"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "Environment": "staging",
            "AZ": "us-west-2a",
            "Version": "1.0",
            "ID": "i-0f87dfd3ae1aedbb2"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "Environment": "staging",
            "AZ": "us-west-2a",
            "Version": "1.0",
            "ID": "i-0f328cea8169f1a11"
        }
    ],
```

```
    [
        {
            "Project": "ERPSystem",
            "Environment": "production",
            "AZ": "us-west-2a",
            "Version": "1.0",
            "ID": "i-0467ec8d7b64aa588"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "Environment": "production",
            "AZ": "us-west-2a",
            "Version": "1.0",
            "ID": "i-04732903097bfe237"
        }
    ]
]
```

Filtramos por las instancias con las etiquetas Project=ERPSystem y Environment=development

aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem" "Name=tag:Environment,Values=development" --query 'Reservations[*].Instances[*].{ID:InstanceId,AZ:Placement.AvailabilityZone,Project:Tags[?Key==`Project`] | [0].Value,Environment:Tags[?Key==`Environment`] | [0].Value,Version:Tags[?Key==`Version`] | [0].Value}'

```
[ec2-user@ip-10-5-0-175 ~]$ aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem"
"Name=tag:Environment,Values=development" --query 'Reservations[*].Instances[*].{ID:InstanceId,AZ:Pl
acement.AvailabilityZone,Project:Tags[?Key==`Project`] | [0].Value,Environment:Tags[?Key==`Environme
nt`] | [0].Value,Version:Tags[?Key==`Version`] | [0].Value}'
[
    [
        {
            "Project": "ERPSystem",
            "Environment": "development",
            "AZ": "us-west-2a",
            "Version": "1.0",
            "ID": "i-0e9426023cbc3d32c"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "Environment": "development",
            "AZ": "us-west-2a",
            "Version": "1.0",
            "ID": "i-0e1c528b2a18d9bc1"
        }
    ]
]
```

Examinamos el archivo *change-resource-tags-sh*

```
[ec2-user@ip-10-5-0-175 ~]$ nano change-resource-tags.sh
```

```
#!/bin/bash

ids=$(aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem"
"Name=tag:Environment,Values=development" --query
'Reservations[*].Instances[*].InstanceId' --output text)

aws ec2 create-tags --resources $ids --tags 'Key=Version,Value=1.1'
```

Este script agrega la etiqueta Version=1.1 a las instancias con etiquetas Project=ERPSystem y Environment=development

Notar que el primer comando usa la opción *--output text* para manipular el resultado como texto en vez de como JSON.

Ejecutamos el script

```
[ec2-user@ip-10-5-0-175 ~]$ ./change-resource-tags.sh
```

Verificamos las etiquetas Version

```
aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem" --query
'Reservations[*].Instances[*].{ID:InstanceId, AZ:Placement.AvailabilityZone,
Project:Tags[?Key==`Project`] | [0].Value,Environment:Tags[?Key==`Environment`]
| [0].Value,Version:Tags[?Key==`Version`] | [0].Value}'
```

```
[ec2-user@ip-10-5-0-175 ~]$ aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSystem"
--query 'Reservations[*].Instances[*].{ID:InstanceId, AZ:Placement.AvailabilityZone, Project:Tags[?K
ey==`Project`] |[0].Value,Environment:Tags[?Key==`Environment`] | [0].Value,Version:Tags[?Key==`Vers
ion`] | [0].Value}'
[
    [
        {
            "Project": "ERPSystem",
            "Environment": "staging",
            "AZ": "us-west-2a",
            "Version": "1.0",
            "ID": "i-0eb8ec00fc8aa4df7"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "Environment": "development",
            "AZ": "us-west-2a",
            "Version": "1.1",
            "ID": "i-0e9426023cbc3d32c"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "Environment": "development",
            "AZ": "us-west-2a",
            "Version": "1.1",
            "ID": "i-0e1c528b2a18d9bc1"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "Environment": "staging",
            "AZ": "us-west-2a",
            "Version": "1.0",
            "ID": "i-0f87dfd3ae1aedbb2"
        }
    ],
    [
        {
            "Project": "ERPSystem",
            "Environment": "staging",
            "AZ": "us-west-2a",
            "Version": "1.0",
            "ID": "i-0f328cea8169f1a11"
        }
    ],
```

## Detener e iniciar recursos con base a las etiquetas

Checamos el contenido del archivo *stopinator.php*

```
[ec2-user@ip-10-5-0-175 aws-tools]$ nano stopinator.php
```

```
#!/usr/bin/php
<?php
# A simple PHP script to start all Amazon EC2 instances and Amazon RDS
# databases within all regions.
#
# USAGE: stopinator.php [-t stop-tags] [-nt exclude-tags]
#
# If no arguments are supplied, stopinator stops every Amazon EC2 and
# Amazon RDS instance running in an account.
#
# -t stop-tag: The tags to inspect to determine if a resource should be
# shut down. Format must follow the same format used by the AWS CLI.
#
# -e exclude-id: The instance ID of an Amazon EC2 instance NOT to terminate. Useful
# when running the stopinator from an Amazon EC2 instance.
#
# -p profile-name: The name of the AWS configuration section to use for
# credentials. Configuration sections are defines in your .aws/credentials file.
# If not supplied, will use the default profile.
#
# -s start: If present, starts instead of stops instances.
# PREREQUISITES
# This app assumes that you have defined an .aws/credentials file.

require 'vendor/autoload.php';
use Aws\Ec2\Ec2Client;

date_default_timezone_set('UTC');

# Obtain the profile name.
$profile = "default";
$opts = getopt('p::t::e::s::');
if (array_key_exists('p', $opts)) { $profile = $opts['p']; }

$excludeID = "";
if (array_key_exists('e', $opts)) {
        $excludeID = $opts['e'];
}

$start = false;
if (array_key_exists('s', $opts)) {
        $start = true;
}

$ec2DescribeArgs = array();
if (array_key_exists('t', $opts))
{
                                                [ Read 124 lines ]
```

```
        $tagArray = array();
        foreach (explode(";", $opts['t']) as $pair) {
                $nameVal = explode("=", $pair);
                array_push($tagArray, array(
                        'Name' => "tag:" . $nameVal[0],
                        'Values' => array($nameVal[1])
                        ));
        }

        $ec2DescribeArgs['Filters'] = $tagArray;
}

# Iterate through all available AWS regions.
$ec2 = Ec2Client::factory(array(
        'profile' =>$profile,
        'region' => 'us-east-1'
));

$regions = $ec2->describeRegions();
foreach ($regions['Regions'] as $region) {
        $instanceIds = array();

        # Find resources in each region.
        echo 'Region is ' . $region['RegionName'] . "\n";
```

```
        foreach (explode(";", $opts['t']) as $pair) {
                $nameVal = explode("=", $pair);
                array_push($tagArray, array(
                        'Name' => "tag:" . $nameVal[0],
                        'Values' => array($nameVal[1])
                        ));
        }

        $ec2DescribeArgs['Filters'] = $tagArray;
}

# Iterate through all available AWS regions.
$ec2 = Ec2Client::factory(array(
        'profile' =>$profile,
        'region' => 'us-east-1'
));

$regions = $ec2->describeRegions();
foreach ($regions['Regions'] as $region) {
        $instanceIds = array();

        # Find resources in each region.
        echo 'Region is ' . $region['RegionName'] . "\n";
        $ec2Current = Ec2Client::factory(array(
                'profile' => $profile,
                'region' => $region['RegionName']
        ));
        $result = $ec2Current->describeInstances($ec2DescribeArgs);
        foreach ($result['Reservations'] as $reservation) {
                foreach ($reservation['Instances'] as $instance) {
                        # Check that this is not an excluded instance.
                        if (strlen($excludeID) > 0 && $excludeID == $instance['InstanceId']) {
                                echo "\tExcluding instance " . $instance['InstanceId'] . "\n";
                        } else {
                                # Is it a running instance?
                                if ($start) {
                                        if ($instance['State']['Code'] == 80) {
                                                array_push($instanceIds, $instance['InstanceId']);
                                                echo "\tIdentified instance " . $instance['InstanceId']$
                                        } else {
                                                echo "\tInstance " . $instance['InstanceId'] . " - not $
                                        }
                                } else {
                                        if ($instance['State']['Code'] == 16) {
                                                array_push($instanceIds, $instance['InstanceId']);
                                                echo "\tIdentified instance " . $instance['InstanceId']$
                                        } else {
```

```
                'profile' => $profile,
                'region' => $region['RegionName']
        ));
        $result = $ec2Current->describeInstances($ec2DescribeArgs);
        foreach ($result['Reservations'] as $reservation) {
                foreach ($reservation['Instances'] as $instance) {
                        # Check that this is not an excluded instance.
                        if (strlen($excludeID) > 0 && $excludeID == $instance['InstanceId']) {
                                echo "\tExcluding instance " . $instance['InstanceId'] . "\n";
                        } else {
                                # Is it a running instance?
                                if ($start) {
                                        if ($instance['State']['Code'] == 80) {
                                                array_push($instanceIds, $instance['InstanceId']);
                                                echo "\tIdentified instance " . $instance['InstanceId']$
                                        } else {
                                                echo "\tInstance " . $instance['InstanceId'] . " - not $
                                        }
                                } else {
                                        if ($instance['State']['Code'] == 16) {
                                                array_push($instanceIds, $instance['InstanceId']);
                                                echo "\tIdentified instance " . $instance['InstanceId']$
                                        } else {
                                                echo "\tInstance " . $instance['InstanceId'] . " - alre$
                                        }
                                }
                        }
                }
        }

        if ($start) {
                if (count($instanceIds) > 0) {
                        echo "\n\tStarting identified instances in " . $region . "...\n";
                        $ec2Current->startInstances(array(
                                "InstanceIds" => $instanceIds
                        ));
                } else {
                        echo "\n\tNo instances to start in " . $region . "\n";
                }
        } else {
                # Stop all identified instances.
                if (count($instanceIds) > 0) {
                        echo "\n\tStopping identified instances in " . $region . "...\n";
                        $ec2Current->stopInstances(array(
                                "InstanceIds" => $instanceIds
                        ));
                } else {
```

```
                                        }
                                }
                        }
                }
        }

        if ($start) {
                if (count($instanceIds) > 0) {
                        echo "\n\tStarting identified instances in " . $region . "...\n";
                        $ec2Current->startInstances(array(
                                "InstanceIds" => $instanceIds
                        ));
                } else {
                        echo "\n\tNo instances to start in " . $region . "\n";
                }
        } else {
                # Stop all identified instances.
                if (count($instanceIds) > 0) {
                        echo "\n\tStopping identified instances in " . $region . "...\n";
                        $ec2Current->stopInstances(array(
                                "InstanceIds" => $instanceIds
                        ));
                } else {
                        echo "\n\tNo instances to stop in " . $region . ".\n";
                }
        }
}
?>
```

El script admite los siguientes argumentos:

- **-t**: un conjunto de etiquetas con el siguiente formato: `name=value;name=value` El script convierte estas etiquetas en el formato que espera la llamada PHP para AWS `Ec2::DescribeInstance()`. Si este parámetro opcional está ausente, el script identificará y cerrará todas las instancias de Amazon EC2 en ejecución en la cuenta.
- **-s**: un parámetro booleano; no requiere argumentos. Cuando este parámetro está presente, las instancias identificadas por **-t** se inician en lugar de detenerse.

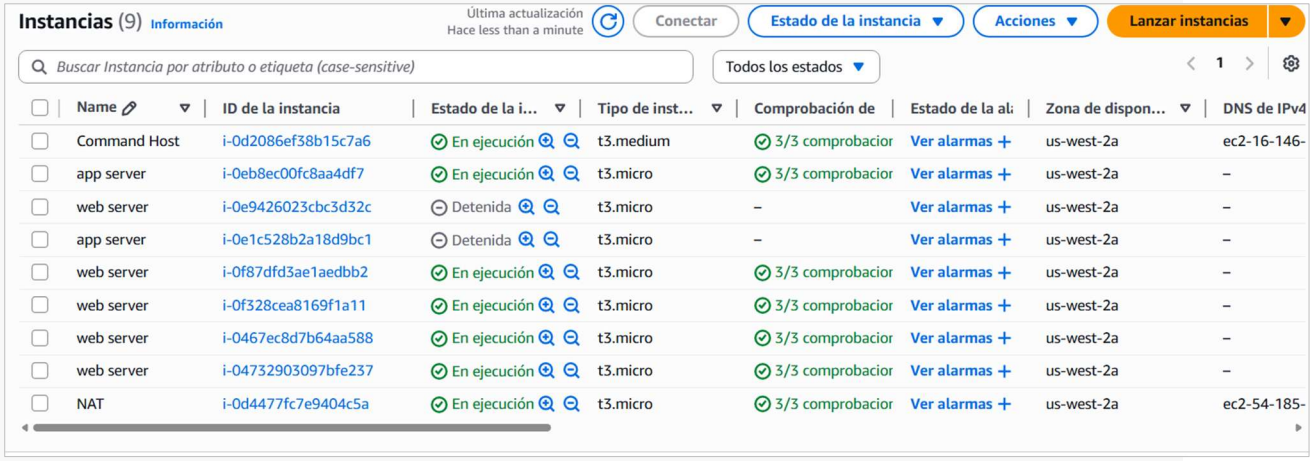
Ejecutamos el script enviando como argumento las instancias con etiquetas Project=ERPSystem y Environment=development

```
./stopinator.php -t"Project=ERPSystem;Environment=development"
```

Notar que si el parámetro -s no se especifica, las instancias será detenidas en vez de ser iniciadas

```
[ec2-user@ip-10-5-0-175 aws-tools]$ ./stopinator.php -t"Project=ERPSystem;Environment=development"
Region is ap-south-1
PHP Notice:  Array to string conversion in /home/ec2-user/aws-tools/stopinator.php on line 120

        No instances to stop in Array.
Region is eu-north-1
PHP Notice:  Array to string conversion in /home/ec2-user/aws-tools/stopinator.php on line 120

        No instances to stop in Array.
Region is eu-west-3
PHP Notice:  Array to string conversion in /home/ec2-user/aws-tools/stopinator.php on line 120

        No instances to stop in Array.
Region is eu-west-2
PHP Notice:  Array to string conversion in /home/ec2-user/aws-tools/stopinator.php on line 120

        No instances to stop in Array.
Region is us-west-2
        Identified instance i-0e9426023cbc3d32c
        Identified instance i-0e1c528b2a18d9bc1
PHP Notice:  Array to string conversion in /home/ec2-user/aws-tools/stopinator.php on line 115

        Stopping identified instances in Array...
```

Las instancias fueron detenidas

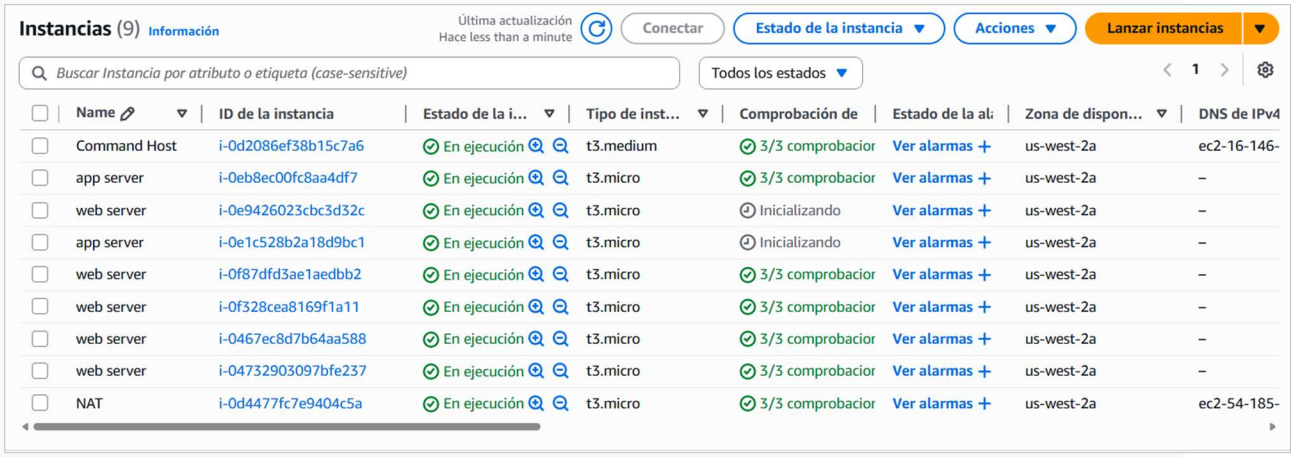



Ahora iniciamos las instancias nuevamente

./stopinator.php -t"Project=ERPSystem;Environment=development" -s

```
[ec2-user@ip-10-5-0-175 aws-tools]$ ./stopinator.php -t"Project=ERPSystem;Environment=development" -s
Region is ap-south-1
PHP Notice:  Array to string conversion in /home/ec2-user/aws-tools/stopinator.php on line 110

        No instances to start in Array
Region is eu-north-1
PHP Notice:  Array to string conversion in /home/ec2-user/aws-tools/stopinator.php on line 110

        No instances to start in Array
Region is eu-west-3
PHP Notice:  Array to string conversion in /home/ec2-user/aws-tools/stopinator.php on line 110

        No instances to start in Array
Region is eu-west-2
PHP Notice:  Array to string conversion in /home/ec2-user/aws-tools/stopinator.php on line 110

        No instances to start in Array
Region is eu-west-1
PHP Notice:  Array to string conversion in /home/ec2-user/aws-tools/stopinator.php on line 110

        No instances to start in Array
Region is us-west-2
        Identified instance i-0e9426023cbc3d32c
        Identified instance i-0e1c528b2a18d9bc1
PHP Notice:  Array to string conversion in /home/ec2-user/aws-tools/stopinator.php on line 105

        Starting identified instances in Array...
```

Las instancias fueron iniciadas



Instancias (9) Información

Última actualización Hace less than a minute | Conectar | Estado de la instancia | Acciones | Lanzar instancias

Buscar Instancia por atributo o etiqueta (case-sensitive) | Todos los estados | 1

| Name | ID de la instancia | Estado de la i... | Tipo de inst... | Comprobación de | Estado de la al: | Zona de dispon... | DNS de IPv4 |
|---|---|---|---|---|---|---|---|
| Command Host | i-0d2086ef38b15c7a6 | En ejecución | t3.medium | 3/3 comprobacior | Ver alarmas + | us-west-2a | ec2-16-146- |
| app server | i-0eb8ec00fc8aa4df7 | En ejecución | t3.micro | 3/3 comprobacior | Ver alarmas + | us-west-2a | – |
| web server | i-0e9426023cbc3d32c | En ejecución | t3.micro | Inicializando | Ver alarmas + | us-west-2a | – |
| app server | i-0e1c528b2a18d9bc1 | En ejecución | t3.micro | Inicializando | Ver alarmas + | us-west-2a | – |
| web server | i-0f87dfd3ae1aedbb2 | En ejecución | t3.micro | 3/3 comprobacior | Ver alarmas + | us-west-2a | – |
| web server | i-0f328cea8169f1a11 | En ejecución | t3.micro | 3/3 comprobacior | Ver alarmas + | us-west-2a | – |
| web server | i-0467ec8d7b64aa588 | En ejecución | t3.micro | 3/3 comprobacior | Ver alarmas + | us-west-2a | – |
| web server | i-04732903097bfe237 | En ejecución | t3.micro | 3/3 comprobacior | Ver alarmas + | us-west-2a | – |
| NAT | i-0d4477fc7e9404c5a | En ejecución | t3.micro | 3/3 comprobacior | Ver alarmas + | us-west-2a | ec2-54-185- |

## Terminar las instancias no conformes

Verificamos el contenido del script *terminate-instances.php*

```php
#!/usr/bin/php

<?php
require 'vendor/autoload.php';
use Aws\Ec2\Ec2Client;

$region = "us-west-2";
$subnetid = "";
$profile = "default"; # Only needed if not using IAM roles

# Necessary to quell a PHP error.
date_default_timezone_set('America/Los_Angeles');

array_shift($argv);
if (count($argv>0)) {
        do {
                $elem = array_shift($argv);
                if ($elem == "-region") {
                        $region = array_shift($argv);
                } elseif ($elem == "-subnetid") {
                        $subnetid = array_shift($argv);
                }
        } while (count($argv) > 0);
}

# Iterate through all available AWS regions.
$ec2 = Ec2Client::factory(array(
        'profile' =>$profile,
        'region' => $region
));

# Obtain a list of all instances with the Environment tag set.
$goodInstances = array();
$terminateInstances = array();

$tagArgs = array();
array_push($tagArgs,  array(
        'Name' => 'tag-key',
        'Values' => array('Environment')
));
$ec2DescribeArgs['Filters'] = $tagArgs;

$result = $ec2->describeInstances($ec2DescribeArgs);
foreach ($result['Reservations'] as $reservation) {
        foreach ($reservation['Instances'] as $instance) {
                $goodInstances[$instance['InstanceId']] = 1;
        }
```

```
}

# Obtain a list of all instances.
$subnetArgs = array();
array_push($subnetArgs, array(
        'Name' => 'subnet-id',
        'Values' => array($subnetid)
));
$ec2DescribeArgs['Filters'] = $subnetArgs;

$result = $ec2->describeInstances($ec2DescribeArgs);
foreach ($result['Reservations'] as $reservation) {
        foreach ($reservation['Instances'] as $instance) {
                echo "Checking " . $instance['InstanceId'] . "\n";
                if (!array_key_exists($instance['InstanceId'], $goodInstances)) {
                        $terminateInstances[$instance['InstanceId']] = 1;
                }
        }
}

# Terminate all identified instances.
if (count($terminateInstances) > 0) {
        echo "Terminating instances...\n";
        $ec2->terminateInstances(array(
```

```
                "InstanceIds" => array_keys($terminateInstances),
                "Force" => true
        ));
        echo "Instances terminated.\n";
} else {
        echo "No instances to terminate.\n";
}
?>
```

Este script toma 2 argumentos: la región y el ID de la subred.

Este script obtiene un arreglo de todas las instancias que tengan la etiqueta "Environment" en la región especificada y un arreglo de todas las instancias de la subred especificada.

Posteriormente compara ambos arreglos y agrega a un nuevo arreglo las instancias que no estén incluidas en la primera lista.

Por último, termina las instancias del tercer arreglo con el método *terminateInstances*.

Para hacer la prueba, eliminamos la etiqueta "Environment" de una instancia

aws ec2 delete-tags --resources i-04732903097bfe237 --tags Key=Environment

```
[ec2-user@ip-10-5-0-175 aws-tools]$ aws ec2 delete-tags --resources i-04732903097bfe237 --tags Key=Envi
onment
```

Las etiquetas también pueden ser administradas desde la consola de administración de AWS

En la pestaña *Etiquetas* de la instancia hacemos clic en *Administrar etiquetas*

Detalles | Estado y alarmas | Monitoreo | Seguridad | Redes | Almacenamiento | Etiquetas

**Etiquetas** — Administrar etiquetas

| Key | Value |
|---|---|
| aws:cloudfor... | Instance6 |
| aws:cloudfor... | c189275a4886476l13832468t1w452228117775 |
| Application | portal |
| Version | 1.0 |
| aws:cloudfor... | arn:aws:cloudformation:us-west-2:452228117775:stack/c189275a4886476l13832468t1w452228117775/f94ad530-0b7b-11f1-9b1a-0a74b4f6dd19 |
| Department | HR |
| cloudlab | c189275a4886476l13832468t1w452228117775 |
| Project | ERPSystem |
| Name | web server |

**Administrar etiquetas** Información

Las etiquetas son rótulos personalizados que se asignan a un recurso de AWS. Puede utilizar etiquetas para organizar e identificar las instancias.

| Clave | Valor - *opcional* | |
|---|---|---|
| Application | portal | Eliminar |
| Version | 1.0 | Eliminar |
| Department | HR | Eliminar |
| cloudlab | c189275a4886476l13832468t1w452228117775 | Eliminar |
| Project | ERPSystem | Eliminar |
| Name | web server | Eliminar |

Agregar nueva etiqueta

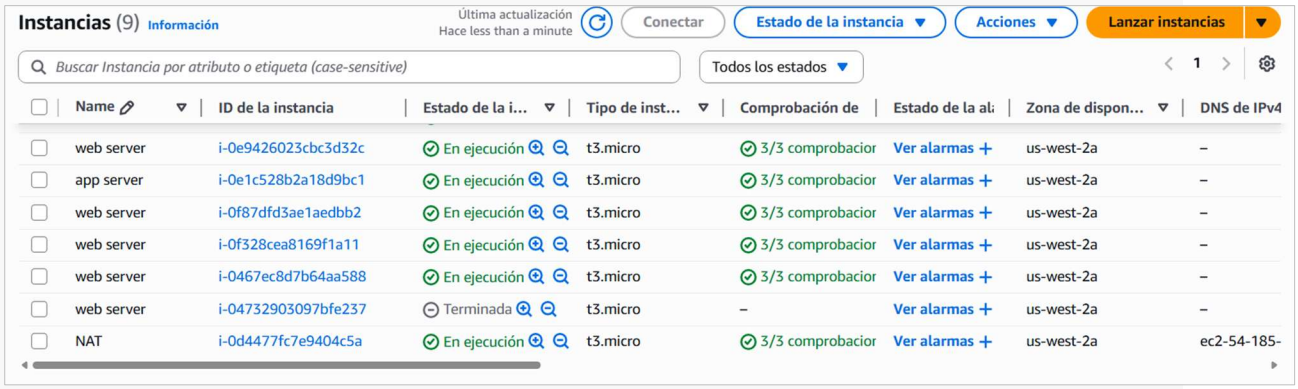
Puede agregar hasta 44 etiquetas más.

Cancelar | Guardar

Ejecutamos el script

./terminate-instances.php -subnetid "subnet-0f66e9e204fff4b34"

```
[ec2-user@ip-10-5-0-175 aws-tools]$ ./terminate-instances.php -subnetid "subnet-0f66e9e204fff4b34"

Checking i-0eb8ec00fc8aa4df7
Checking i-0e9426023cbc3d32c
Checking i-0e1c528b2a18d9bc1
Checking i-0f87dfd3ae1aedbb2
Checking i-0f328cea8169f1a11
Checking i-0467ec8d7b64aa588
Checking i-04732903097bfe237
Terminating instances...
Instances terminated.
```

En este caso omitimos el argumento de la región porque es la misma que tiene por default en el código PHP (us-west-2)

Se ha detenido la instancia con el id especificado